

3D reconstruction of a static scene from multi-view

Dinh Minh NGUYEN¹ and Luong Phuong Truc HUYNH¹

¹Université Jean Monnet

Professor: Alain TREMEAU, Instructor: Isidore DUBUISSON

Summary—This work studies and implements three approaches to depth reconstruction from static images. We start with stereo visions where object depth is inferred from disparity maps transformed using pairs of images. We then incorporate a third camera head and discuss a trifocal matching method based on template matching. Finally, we experiment with DUST3R - a state-of-the-art deep-learning approach to depth reconstruction. Observations, discussions, and comparisons will be made along the way.

Keywords—Stereo vision, multi-view reconstruction, point cloud

Contents

1	Introduction	1
2	Image acquisition and calibration patterns	2
2.1	Calibrate with chessboards	2
2.2	Calibrate with ChArUco boards	3
3	Camera calibration	3
3.1	Match image points across views	3
3.2	Estimate intrinsic parameters	4
4	Stereo setup	4
4.1	Estimate fundamental matrices	5
4.2	Estimate camera relative poses	5
4.3	Visualize epipolar constraints	5
4.4	Rectify a stereo setup	6
4.5	Compute disparity maps	7
4.6	Reconstruct a dense point cloud	7
5	Three-view setup	8
5.1	Estimate pair-wise relative poses	8
5.2	Find correspondences with template matching	8
5.3	Reconstruct a sparse point cloud	9
5.4	Reconstruct local dense point clouds	9
6	Point cloud reconstruction with DUST3R	10
6.1	A brief overview of DUST3R	10
6.2	State-of-the-art point cloud reconstruction	10
7	Comparisons and conclusion	11

1. Introduction

3D point cloud reconstruction from static images is a critical area of research and development in computer vision, with significant applications in fields such as autonomous driving, robotics, and virtual reality. This process involves converting 2D images into 3D representations, allowing a more detailed and accurate perception of the physical world. Generally, there are many techniques to address this problem, each offering certain advantages and disadvantages [6].



Figure 1. The scene used for reconstruction

This work explores three distinct approaches to depth reconstruction from static images. We begin with stereo vision in Section 4, a well-established technique that leverages disparity maps derived from image pairs to estimate object depth. This method relies on the parallax principle where the displacement of objects when viewed from different angles provides crucial depth information. Disparity maps thus can be computed by appropriate block matching procedures, resulting in dense 3D reconstruction of the scene.

We then introduce a trifocal system in which a third camera is incorporated. We leverage epipolar constraints across three views and employ template matching for this setup to find correspondences. Depth inference thereby is more reliable and the hyperparameter space is reduced significantly. We also implement a Python script to interact and verify the matching at runtime, enabling even more fine-tuned results. Section 5 is devoted to this multi-view reconstruction.

Finally, Section 6 experiments with DUST3R [7], a cutting-edge deep learning model designed specifically for depth reconstruction. This state-of-the-art approach harnesses the power of neural networks to infer depth from a single image, bypassing the need for traditional mathematics-based methods. DUST3R leverages large datasets and advanced training techniques to produce highly accurate depth maps, pushing the boundaries of 3D point cloud reconstruction.

Before getting into the methods, we will spend Section 2 discussing the image set acquired from three identical cameras and how image points are reliably matched. We will discuss issues arising in these processes and methods to overcome them. Section 3 gives implementation details of the said approach and lists the estimated intrinsic parameters and their associated re-projection errors. In the end, Section 7 provides final comparisons and possible future improvements.



Figure 2. The views, in left-to-right order, as seen by the left, front, and right cameras, respectively.

2. Image acquisition and calibration patterns

This section first presents the setup for static image acquisition. The work uses three identical GoPro cameras placed at three locations and with different orientations to capture the scene shown in Figure 1.

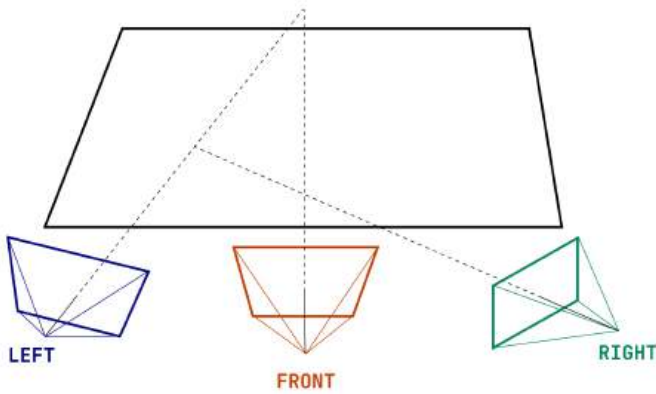


Figure 3. Camera setups for image acquisition

The positions and orientations of the cameras concerning the table are roughly illustrated in Figure 3. The scene views captured by these cameras are shown in Figure 2.

We shall explain this choice of acquisition. The intention was to encapsulate as many cases as possible in just one image set, in particular:

- The front camera is positioned at the scene center, looking down at the objects.
- The left camera is placed at the same height as the front one, oriented so that its line of sight meets the front's as illustrated in the figure.
- The right camera, on the other hand, is placed at a lower height, oriented toward the far left side of the scene.

With this acquisition setup, the left-front pair closely represents a standard stereo vision, while the right camera collectively emulates a random multi-view setup. This way, we can selectively apply our methods to different setup pairs and observe the differences.

2.1. Calibrate with chessboards

Part of the reconstruction process is finding reliable correspondence sets between views to calibrate the camera's intrinsic

and extrinsic parameters. In practice, such correspondences can be programmatically found with established methods like SIFT [2] or ORB [4].

Despite being widely used, these methods might produce unreliable matches given our fairly texture-lacking objects. Such false matches could greatly affect the calibration quality, which, in turn, harms the reconstruction outcome.

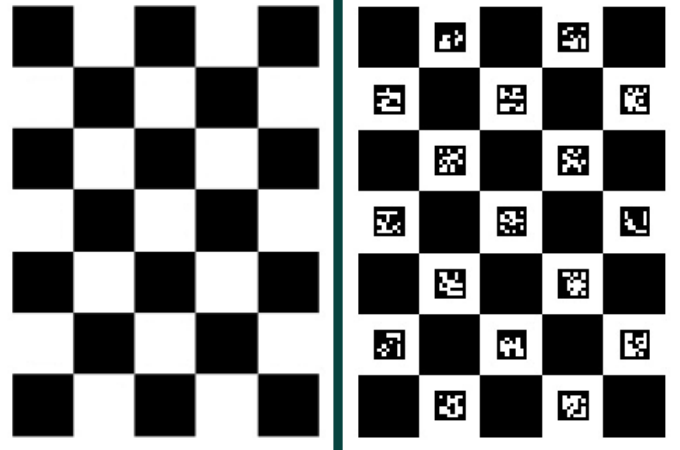


Figure 4. Popular calibration patterns for correspondence matching. (Left) Chessboard. (Right) Chessboard with ArUco markers.

Our workflow thus extensively employs distinct calibration patterns to maximize the estimation quality of intrinsic and extrinsic parameters. Figure 4 (Left) depicts one of the popular choices of such patterns - a plain chessboard.

When a chessboard is placed in the scene, most computer vision frameworks would have little trouble detecting the image coordinates of the chessboard corners, regardless of its orientation to the camera. Unless positioned poorly, it is expected to detect 24 image coordinates for the chessboard shown in Figure 4. Effectively, when a camera pair captures a scene containing the board, we thus recover 24 image correspondences.

Unfortunately, 24 matches might not be enough for parameter estimation to converge to an optimal solution. Methods like SIFT or ORB could detect a few hundred matching pairs in one go, compensating for any noises and outliers presented in the captures.

Because detections of chessboard corners are extremely precise, we may only need around 100 matches as opposed to SIFT or ORB. To achieve this, a widely used solution is progressively altering the chessboard orientation and position between captures. This way, 4 calibration acquisitions would already result



Figure 5. Uneven matches between images and how ArUco markers can help retain detected corners. Because the right camera has a poor view of the ChArUco board, it cannot detect the hard corners in the bottom left. The left view, despite being able to detect the full corner set, drops only the missing corners using unique IDs.

in 96 correspondences. This number is possible because the likelihood of having the same corner sets between different alternations is improbable.

However, this scheme comes with a major setback. While altering the board between takes, some cameras might get a better view of the patterns than others. This is unavoidable because we want to vary the calibration patterns greatly, otherwise most correspondence pairs would be the same. This leads to situations where we might detect a full chessboard corner set in one image but only a partial number of them in another. In other words, the detected corners may not be equal between the same acquisition pair.

2.2. Calibrate with ChArUco boards

When the stated issue happens, there is no choice but to discard the entire acquisition pair because we cannot feasibly differentiate which corners from one image correspond to which other corners in another image. If retaining the commonly detected corners is desirable, one may replace plain chessboards with chessboards containing ArUco markers [5], as shown in Figure 4 (Right).

In a nutshell, ArUco markers are binary square fiducial markers that act like unique fingerprints for the detected corners. As a result, we can filter out commonly detected corners since each of them now has a unique ID. A chessboard overlaid with ArUco markers is thus called a ChArUco board.

Figure 5 demonstrates the discussed situation where the detector in the right image could not identify the 6 remaining corners in the bottom left of the board. Because we know the unique IDs of all detected corners, we thus discard those 6 corners identified in the left image, retaining the common 18 corners for the subsequent parameter estimation.

3. Camera calibration

This section first gives detailed steps to find the corresponding image coordinates between a camera pair. It then demonstrates how the discussed procedure can recover camera intrinsic parameters and their associated re-projection errors.

3.1. Match image points across views

The matching process can be generalized into a generic procedure for convenient reusability. The implementation takes advantage of the CharucoDetector class from opencv's aruco module. The class exposes a detectBoard and a matchImagePoints method. The former detects and returns

corners with their associated IDs given an image containing a ChArUco board, while the latter prepares 3D object points and 2D image points from the detected corners.

Algorithm 1: match_stereo_charuco

Input:

- l_{images} - an array of n images from the left camera
- r_{images} - an array of n images from the right camera
- `detector` - an instance of CharucoDetector

Output:

- o_{sets} - an array of at most n arrays, each containing the matched object points from each image
- l_{sets} - an array of $k \leq n$ arrays, each has the matched image points from each image in l_{images}
- r_{sets} - an array of $k \leq n$ arrays, each has the matched image points from each image in r_{images}

$o_{\text{sets}} \leftarrow []; l_{\text{sets}} \leftarrow []; r_{\text{sets}} \leftarrow [];$

for $i \leftarrow 0$ to n do

$l_{\text{image}} \leftarrow l_{\text{images}}[i];$
 $r_{\text{image}} \leftarrow r_{\text{images}}[i];$

$l_{\text{corners}}, l_{\text{ids}} \leftarrow \text{detector.detectBoard}(l_{\text{image}});$
 $r_{\text{corners}}, r_{\text{ids}} \leftarrow \text{detector.detectBoard}(r_{\text{image}});$

Find common IDs

$\text{common} \leftarrow \text{intersect}(l_{\text{ids}}, r_{\text{ids}});$

if $\text{size}(\text{common}) \leq 0$ **then**
 | **continue**

$l_{\text{corners}} \leftarrow l_{\text{corners}}[\text{common}];$
 $r_{\text{corners}} \leftarrow r_{\text{corners}}[\text{common}];$
 $c_{\text{ids}} \leftarrow l_{\text{ids}}[\text{common}];$

$\text{board} \leftarrow \text{detector.getBoard}();$
 $o, l_p \leftarrow \text{board.matchImagePoints}(l_{\text{corners}}, c_{\text{ids}});$
 $o, r_p \leftarrow \text{board.matchImagePoints}(r_{\text{corners}}, c_{\text{ids}});$

$o_{\text{sets}}.append(o);$

$l_{\text{sets}}.append(l_p);$

$r_{\text{sets}}.append(r_p);$

end

return $o_{\text{sets}}, l_{\text{sets}}, r_{\text{sets}}$

Algorithm 1 illustrates a reliable procedure to pick up common corners from a set of image pairs using an instance of CharucoDetector. Each image pair captures an alternation of the ChArUco board as seen from a left and a right camera.

The function returns three array sets containing the matched object points and their corresponding left and right image points. Each array in the sets corresponds to one of the input image pairs. The total size of these arrays from each set is the collective number of matches that can be used for subsequent estimation steps.



Figure 6. Performing `match_stereo_charuco` on each camera pair. From top to bottom: front-right, left-front, left-right pair.

Figure 6 shows the results obtained from the discussed procedure where only an image pair is included for each camera pair. An actual run of Algorithm 1 outputs many such pairs, depending on the input arguments.

To make the results more convincing, the corners detected in one image are colored according to their match in another. This way of visualizing helps validate the function outputs and avoid any unexpected bugs.

3.2. Estimate intrinsic parameters

The obtained object points and their corresponding image points from a camera can be used to recover its intrinsic parameters, including the focal length in the x and y axes. These focal lengths are measured in pixel units and directly proportional to the size of input images.

The calibration was performed with the help of `opencv's` `calibrateCamera` function. Under the hood, the function conducts several mathematical and optimization steps to achieve accurate camera calibration. Once done, it returns a K matrix representing intrinsic parameters, a set of distortion coefficients for undistorting the input image, and the total reprojection error.

The reprojection error is calculated by using the computed K to project the provided object points onto the image and the results are compared with the actual image points. Generally, a total reprojection error of less than 0.5px is preferable.

Table 1 details the estimated intrinsic parameters returned by `calibrateCamera`. These parameters result from input images of size 5568×4176 and are relatively close to each other. While the front and right cameras have the preferred reprojection errors, the left camera parameters provide an error bigger than 0.5. This is acceptable nonetheless, given big-size input images.

	Front Cam	Left Cam	Right Cam
f_x (px)	2580	2536	2657
f_y (px)	2580	2514	2646
r_{error} (px)	0.413	0.528	0.490

Table 1. Estimated focal lengths in x and y axes and their reprojection errors.

4. Stereo setup

This section provides the detailed steps to reconstruct 3D point clouds based on stereo vision. Prerequisites to this process are calibrated intrinsic parameters and their distortion coefficients found in Section 3. In addition, the image correspondences (corners) found on ChArUco boards from the image pairs also contribute to this process.

Although this section extensively uses the terms "left" and "right" cameras, they do not designate the left and right cameras in our acquisition setup. Rather, these identifiers address general camera heads responsible for a stereo system's left and right views. In other words, the left-front, front-right, and left-right camera pairs in our setup are valid inputs to the workflow discussed in this section.

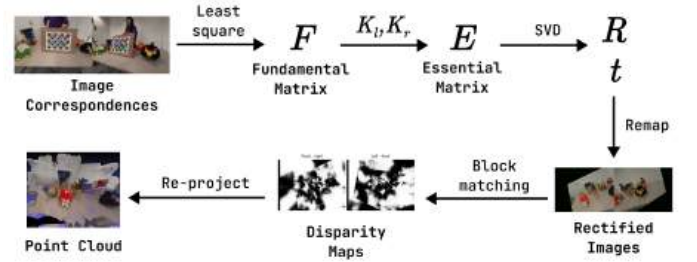


Figure 7. Abstract workflow of stereo reconstruction

Figure 7 sketches the overview workflow of our stereo reconstruction implementation. The whole process is based on a simple equation that relates the depth, Z , of a 3D point, say P , to its disparity observed from a stereo pair:

$$Z = \frac{Bf}{d} \quad (1)$$

Here, B denotes the baseline distance between the two cameras measured in world-length units, while f denotes the camera focal length measured in pixels. The term d in the denominator measures the disparity of the 3D point projected on the stereo pair, again in pixel units. Specifically, if u_l and u_r denote the image x -coordinates of P on the left and right image, the disparity of P is effectively $u_l - u_r$.

Given Equation 1, if we can compute the disparity for each feature point at image pixel (u, v) , we will effectively reconstruct the depth of the whole scene. The x and y coordinates in world space where the coordinate system is placed at the camera center (i.e. camera space) can then be calculated based on the following relations:

$$x = \frac{uZ}{f_x}, \quad y = \frac{vZ}{f_y}$$

We gloss over the possibility that u, v might be defined in pixel coordinates where the image origin is located at the top left corner. However, such information is encoded in the last

column of the intrinsic matrix K which is instead used in practice to recover the world coordinates of P .

An image encoding the disparity values of the observed object at every pixel is thus known as a *disparity map*. The most well-known method to efficiently compute such maps is to perform *block matching* on the stereo pair's rectified version.

Rectification thus becomes a crucial operation in our stereo reconstruction pipeline, transforming the stereo pair so that their epipolar lines are aligned and parallel to the baseline. This operation demands the knowledge of the cameras' relative poses, meaning the rotation matrix R and the translation vector t that bring coordinates in one camera to another:

$$\begin{bmatrix} x^{(l)} \\ y^{(l)} \\ z^{(l)} \\ 1 \end{bmatrix} = \begin{bmatrix} R & t \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} x^{(r)} \\ y^{(r)} \\ z^{(r)} \\ 1 \end{bmatrix} \quad (2)$$

The superscript l and r designate the world coordinates of P relative to the left and right camera's coordinate frame, respectively. Eventually, the world coordinate frame will be either the two camera frames. If R and t satisfy Equation 2:

- R will be the orientation of the **left** camera in the **right** camera's coordinate frame.
- t will be the position of the **right** camera in the **left** camera's coordinate frame.

In particular, the rows of R are the coordinates of the left camera's $\hat{i}$, $\hat{j}$, and $\hat{k}$ vectors in the right camera's coordinate frame. Suppose $t = (t_x, t_y, t_z)$, we define a skew matrix:

$$T_X = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

If we let $E = T_X R$ be the *essential matrix* between the left and right cameras, we arrive at a constraint for pairs of 3D correspondences:

$$\begin{bmatrix} x^{(l)} & y^{(l)} & z^{(l)} \end{bmatrix} E \begin{bmatrix} x^{(r)} \\ y^{(r)} \\ z^{(r)} \end{bmatrix} = 0 \quad (3)$$

To convert this constraint for use with image coordinates, we define the *fundamental matrix*:

$$F = (K^{(l)})^{-1} E (K^{(r)})^{-1} \quad (4)$$

The constraint in Equation 3 effective becomes compatible with image correspondences detected in previous steps:

$$\begin{bmatrix} u^{(l)} & v^{(l)} & 1 \end{bmatrix} F \begin{bmatrix} u^{(r)} \\ v^{(r)} \\ 1 \end{bmatrix} = 0 \quad (5)$$

Because we have many such correspondences (found with ChArUco detections in Section 3), the problem of finding F thus becomes an over-constrained problem that can be solved with linear least squares. Therefore, these correspondences must be preferably accurate, a tight condition assured by ChArUco boards.

4.1. Estimate fundamental matrices

Instead of performing least square from scratch, we leverage opencv's `findFundamentalMat(p1, p2, ...)` function. Here, p_1 and p_2 are lists of floating-point image correspondences. The function returns a matrix F described by the following equation:

$$\begin{bmatrix} p_2 & 1 \end{bmatrix} F \begin{bmatrix} p_1 \\ 1 \end{bmatrix} = 0$$

Therefore, if we want to find F as described in Equation 5, we need to assign p_1 and p_2 as the right and left image corners, respectively. The function provides 4 estimation methods other than least square, among which we found that the 8-point algorithm gave reasonable results.

4.2. Estimate camera relative poses

Equation 4 can be rewritten to find the essential matrix E once we have estimated F :

$$E = K^{(l)T} F K^{(r)}$$

To recover the rotation matrix and the translation vector, we take advantage of the fact that E is a product of a skew-symmetric matrix (T_X) and an orthonormal matrix (R). Such a matrix can be decomposed with Singular Value Decomposition, giving three matrices U , W , V^T such that $E = U W V^T$.

Based on the procedure described by [1], the rotation matrix and the translation vector making up E are one of the following:

- $[R|t] = [U W V^T \mid U [0 \ 0 \ +s]^T]^T$
- $[R|t] = [U W V^T \mid U [0 \ 0 \ -s]^T]^T$
- $[R|t] = [U W^T V^T \mid U [0 \ 0 \ +s]^T]^T$
- $[R|t] = [U W^T V^T \mid U [0 \ 0 \ -s]^T]^T$

Here, W is defined as $\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, and s is a positive scaling

factor. To summarize, given an essential matrix E , two opposite directions are possible for t and two different R compatible with this essential matrix. This gives four solution classes for the rotation and translation between the two camera coordinate systems. On top of that, there is also an unknown scaling $s > 0$ for the chosen translation direction.

It turns out, however, that only one of the four classes can be realized in practice. Given a pair of corresponding image coordinates, three solutions will always produce a 3D point lying behind at least one of the two cameras and therefore cannot be seen. Only one of the four classes will consistently produce 3D points in front of both cameras. This must then be the correct solution. Still, however, it has an undetermined positive scaling related to the translation component.

4.3. Visualize epipolar constraints

Because the fundamental matrix is the starting point for all subsequent computations, we draw epipolar constraints on image pairs to verify the correctness of F . While Equation 5 gives an

immediate formulation of the epipolar lines' coefficients, we leverage `opencv's computeCorrespondEpilines()` function and follow the same coloring scheme as described in previous sections.

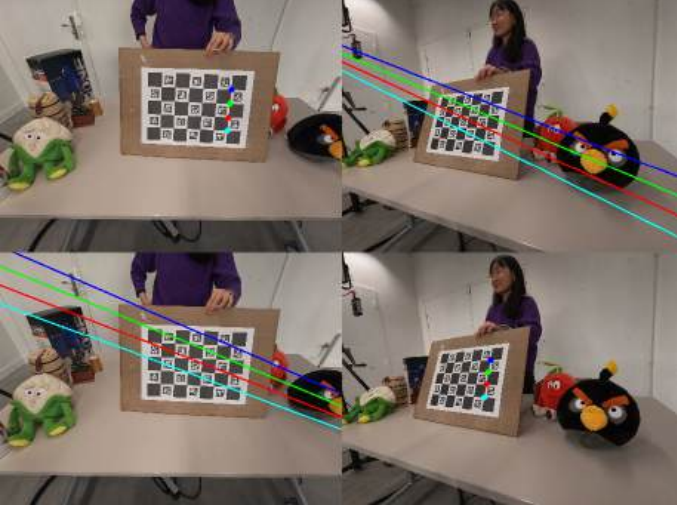


Figure 8. Detected corners and their corresponding epipolar lines between the front (left column) and right (right column) camera heads.

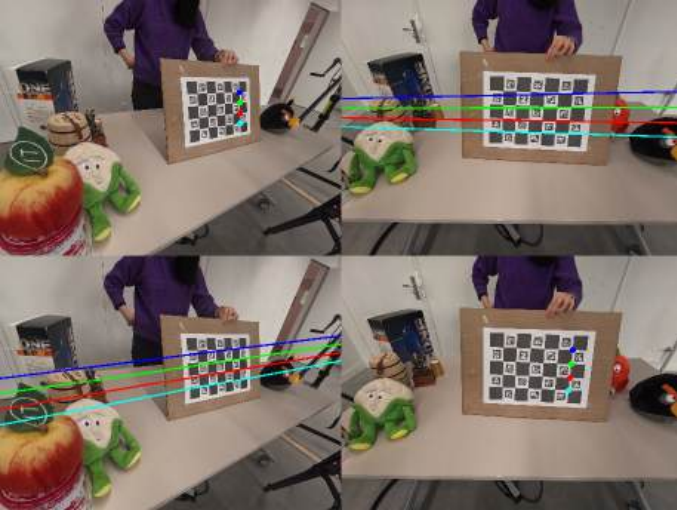


Figure 9. Detected corners and their corresponding epipolar lines between the left (left column) and front (right column) camera heads.

Figure 8 illustrates the said idea. The epipolar lines corresponding to the detected corners from the front view are drawn on the right view, and vice versa. We only pick four detected corners in the drawing to avoid cluttering the visualization.

These plots reveal a convincing estimation of the fundamental matrix. Specifically, all epipolar lines in one view cross their matching correspondence in another view. They also meet at the projected point of the other camera. Because the right camera was placed at a lower height than the front one, the epipolar lines thus tilt more pronounced in this case.

On the other hand, the epipolar lines illustrated in Figure 9 for the left and front views lie more horizontally flat. As stated in Section 2, the left camera was intentionally placed at roughly the same height as the front camera. As a result, the epipolar lines drawn for this pair are more parallel to the image's horizontal edges.

4.4. Rectify a stereo setup

The epipolar lines visualized in the previous section reveal that a point in one image and its matching in another are not horizontally aligned. This causes trouble for the block matching algorithm in which correspondences are expected to lie on the same line parallel to the horizontal edges.

This raises a need for rectification, an image transform process that makes all epipolar constraints parallel and perfectly aligned. It also makes the baseline of the stereo pair horizontal, allowing block-matching to search horizontally.

With `opencv's` rich support for image processing, the rectification was done with the `initUndistortRectifyMap()` and `remap()` functions. The former expects the stereo pair relative poses, i.e. the rotation matrix and the translation vector, to compute mapping matrices along the x and y directions. The latter, in turn, takes those mappings and applies 2D transformations to the images.



Figure 10. Rectified images of the front and right view



Figure 11. Rectified images of the left and front view

Figures 10 and 11 show rectified front-right and left-front image pairs based on the relative poses identified in the previous section. The transform brings all matching features to the same height and rotates the images as if the two cameras were placed at the same level.

Since the rectified images are larger than their original sizes, we limit the image size to the original dimensions in our implementation. As a result, some details are discarded; for example, the black angry bird on the left side of Figure 10 is excluded. Consequently, this object won't be correctly matched to calculate disparity, leading to possible reconstruction failure in subsequent steps. Ensuring feature alignment and retaining details after rectification is crucial for successful results.

In the case of the rectified image of the left and front view in Figure 11, the alignment is similarly precise. However, the significant viewpoint change imposes a challenge. The left image prominently features the left side of Ganesh and the black angry-bird, which are markedly different from the front view in the right image.



Figure 12. Stereo reconstruction from the front and left view



Figure 13. Stereo reconstruction from the left and right view

4.5. Compute disparity maps

With the rectified images ready, an instance created by `opencv's StereoSGBM_create` shall compute disparity maps. It initializes the stereo Semi-Global Block Matching (SGBM) algorithm [3], which is widely used for computing disparity maps from stereo image pairs.

While the method offers robust results, it requires customization of a large parameter set like block size, disparity range, and pre-filtering to fine-tune the performance for specific images and applications. As a result, we had to spend time tweaking and testing these parameters for visually acceptable outputs.

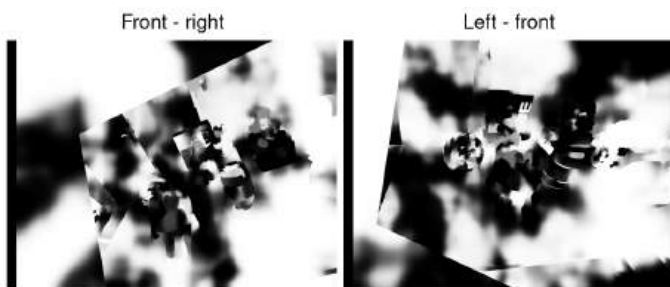


Figure 14. Disparity maps from the front-right and left-front views

Figure 14 depicts the disparity maps of the front-right and left-front views. As mentioned earlier, most parts of the black angry bird are missing in the front rectified image. This failure is evident in the front-right pair where the object appears as a white blank area on the right of the disparity map, while other objects' shape and depth in the scene are visible.

However, the disparity map of the left-front view, affected by large viewpoint changes, appears noisy with unrecognizable shapes and depths of objects blending into the background. This could result in a noisy reconstruction scene.

4.6. Reconstruct a dense point cloud

In the previous sections, we analyzed the rectified images and disparity maps to predict the quality of the reconstructed scene. We can now finalize our computations with the reconstruction results shown in Figures 12 and 13. This was made possible thanks to `opencv's reprojectImageTo3D()` function and the `open3d` library.

In the reconstruction based on the front-right view, the black angry bird failed to be constructed, but other objects are well realized. The Ganesh statue is partially reconstructed by our implementation, albeit only half of its view on the left due to significant viewpoint changes. As we shall see, this object might cause trouble for state-of-the-art reconstruction models to recover, suggesting that traditional methods based on stereo vision still hold their place in this area. Larger objects like the red stuffed animal and the small wooden crate exhibit smooth surface curves in the reconstruction, thanks to their size and distinctive colors compared to the background and varying color shading within the objects.

The reconstruction from the left-front view is noisy nonetheless due to the dramatic viewpoint change. Notably, the apple and its pedestal stand out without noise and remain intact, thanks to their unique and uniform appearance, making them easily distinguishable from the background and unaffected by viewpoint changes. However, the apple appears flat in the reconstruction as the diffuse texture makes shading for depth interpretation challenging.

In short, we gained initial insights into 3D reconstruction based on stereo vision. For better reconstruction results, we should choose objects with unique features or textures that are not diffuse and ensure their locations are not too occluded from each other. For scene acquisition, cameras should not be placed too far apart to avoid large viewpoint changes.

5. Three-view setup

This section describes how we incorporated a third camera head into our reconstruction solution. In the stereo vision case, we have seen that failed matches led to unsatisfied results. In other words, it is the dense matching quality that defines the reconstruction outcome. Therefore, our idea when incorporating the third camera is to impose more clues for the matching step. The hope is that this other constraint will guide the matching algorithm in detecting more accurate correspondences, producing more visually appealing point clouds.

5.1. Estimate pair-wise relative poses

The calibration process in the three-view setup is identical to what has been exercised in previous sections. Specifically, we will use the same calibration patterns, intrinsic estimations, and relative pose computations. The only difference is that the rotation matrix and translation vector are derived separately for all camera pairs, the front-right, front-left, and left-right.

Stereo pair	Reprojection error (px)
Front-right	0.2693
Front-left	0.3111
Left-right	0.3820

Table 2. Re-projection errors of each stereo pair, using the estimated rotation matrix and translation vector between pair-wise heads

Table 2 details the pair-wise reprojection errors of the setup. Given a stereo pair, its estimated relative poses are used to project the object points as seen from one camera on another. The projected points are compared with the expected image points and the errors are accumulated. The table reveals that the left-right pair has the highest error, which is reasonable given its dramatic shift in viewing direction. Nevertheless, these errors are preferably small, shouting out to the ChArUco calibration patterns.

5.2. Find correspondences with template matching

We start discussing our 3-view reconstruction method. Recall that the goal is to find reliable matches in two other images given any point in one image. For example, for one of the black angry bird's eyes, as illustrated by the green dot in the front view in Figure 15, we expect to find the pixel coordinates in the left and right view corresponding to this point. More precisely, the respective red and blue points in the top left and right images where the eye is seen from these views.

One of the algorithms to find such correspondences is known as *template matching*. Consider a small window placed at the green dot. To search for this point's correspondence in some other view, we use another same-size window and march it across the new image, calculating value differences between the pixels within the two windows. The best matching candidate would be the pixel location where the difference is minimal among its possible locations in the second image.

There are numerous metrics to compute pixel differences between the two windows. The easiest choice among them is to calculate the sum of squared differences between their pixel values. The sum can be normalized to the range $[0, 1]$, in which case the metric becomes the normalized sum of squared differences. Other well-known metrics are (normalized) cross-correlation and (normalized) correlation coefficient.



Figure 15. Multi-view epipolar constraints and the correspondences found with template matching. The bottom image is from the front camera where the starting point is sampled (green dot), while the top left and right images are from the left and right cameras. The green epipolar lines are associated with the green dot in the front view, while the blue line in the left view is associated with the blue dot in the right view.

Given that the correspondence is based solely on local pixel values and the marching window is free to move across the whole image, one may suspect this method's quality and efficiency. The good news is we have epipolar constraints at our disposal, thereby restricting the space where we conduct template matching. Recall that for any point in one image, epipolar geometry tells us that its correspondence must lie on a line in another view. This reduces our template matching's search space to 1D, substantially improving the method's performance.

However, it is still not guaranteed that the local window where we compute the matching metric will give reliable results even along the epipolar line. Nothing prevents a scene from having distinct pixel values across any given line. This is where we use the third view to constrain further the search. Specifically, for each candidate window location along the epipolar line in the second image, we immediately have another epipolar line for the third view. This line intersects the epipolar line imposed by the sampled point in the first image, giving us another window location for template matching. Generally, for a multi-view setup, any two points from any two views will immediately constrain their correspondences on the remaining views.

Given that we now have two local windows to match against the template in the front view, we will calculate the error metrics for both windows and sum them up to get the final error. This idea is borrowed from trifocal tensor theory where each view pair contributes to the correspondence matching. Generally, for a multi-view setup involving n views, the matching

shall be computed $\binom{n}{2}$ times for each pair. This way, the view from the third camera will contribute to the quality of the dense correspondence problem. Moreover, we can further restrict the search along the line segment where the intersection of the two epipolar lines in the third view lies inside the image.

Once we have found the matches in the left and right view of the sampled point, we can select either correspondence and perform triangulation with the sample to recover 3D coordinates. The triangulation depends on the relative poses between the camera pairs whose correspondences were chosen and their intrinsic matrices. We leverage `opencv's` `triangulatePoints()` function to facilitate the implementation. Under the hood, it performs yet another least square optimization to find the 3D coordinates of the sample.

One may ask why template matching of all sophisticated methods. The answer is that we want to search manually on non-horizontal epipolar lines, and template matching is the easiest starting point to implement with less error-prone. The problem with `opencv's` `matchTemplate()` function is that it always performs the search on the whole image, drastically increasing the computation time.

5.3. Reconstruct a sparse point cloud

We're now ready to put the method into action. Initially, we only reconstruct a selective number of points where the found matches are visually correct, similar to the result shown in Figure 15. Because this method is nonetheless underdeveloped, it's better to start with small samples and observe the behavior.

To streamline this process, we developed an interactive program that shows three views simultaneously and has users select sampled points in the front view. Each click into the front window will immediately result in the computed match in two other images, calculated based on the discussed method. Users can then hit SPACE to push the current sample into the triangulation process if they are satisfied with the computed correspondence.

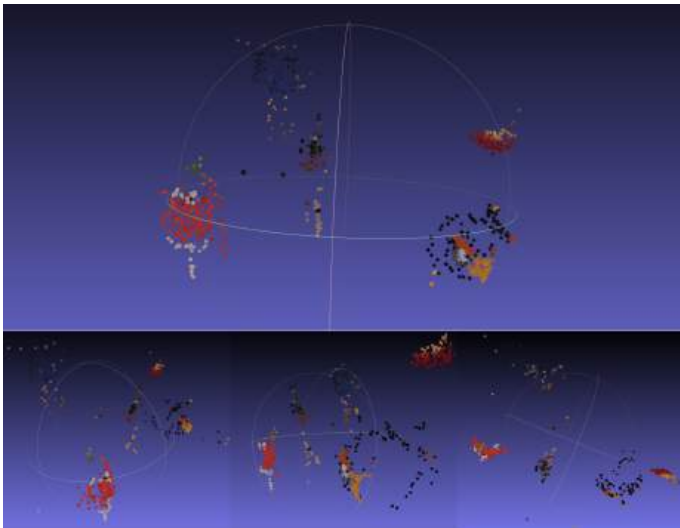


Figure 16. Sparse reconstruction using 3-view epipolar constraints and template matching. The top image is the bird-eye view of the reconstruction, while the bottom three are the same point cloud viewed from different angles.

Figure 16 shows the reconstruction result following the said strategy. Because it is infeasible to densely sample points by hand, the result is thus a *sparse* point cloud. Nevertheless, we

can observe features and objects that emerged from the 3-view method. The bottom row of the figure illustrates the same point cloud but at different viewing angles. We can see that the objects are reconstructed to the right depth as perceived in the real scene. This verifies our triangulation and color back-projection correctness.

5.4. Reconstruct local dense point clouds

Having verified the depth and color back-projection correctness, we can confidently reconstruct dense point clouds using the discussed method. The interactive program mentioned earlier also allows users to select a region of interest (ROI) by specifying a set of points enclosing it. Once accepted, the program masks all pixel coordinates outside the ROI and performs the 3-view match method on the in-ROI pixels.



Figure 17. Dense reconstructions with 3-view setup at local windows. Each row is the same local point cloud viewed from three different angles.

Figure 17 shows several point clouds reconstructed from different ROIs. Because they only account for a subset of the scene, these reconstructions are thus coined local dense point clouds. Each row of the figure is a reconstructed object appearing in the scene, viewed from different angles.

There are several points worth discussing regarding these local reconstructions. For the most part, these point clouds are

visually appealing enough to give motivations for improving the method. We can see the clear appearance of the objects, all projected to the acceptable depth. However, we couldn't recover the whole surface of these objects. This is attributed to the lack of surface texture, causing the template matching to find suboptimal correspondences.

Another aspect where our method exhibits deficiency is that we couldn't recover objects whose colors match the background color like the cauliflower or the base of the Ganesh statue. This suggests a more sophisticated matching algorithm should be employed to decouple the color feature of the object and the scene background. Nevertheless, our method still manages to reconstruct acceptable point clouds for a fairly simple matching algorithm.

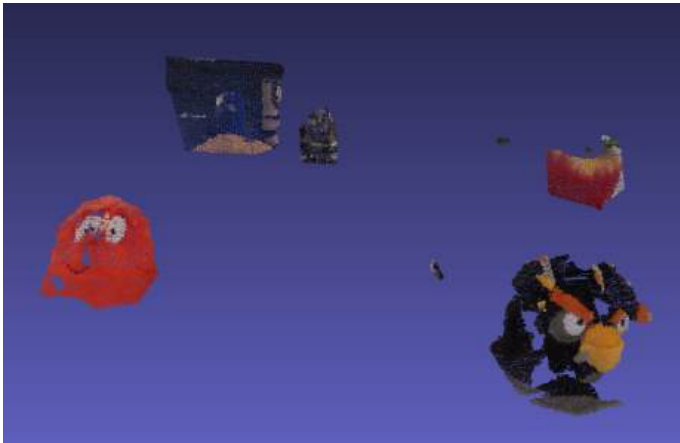


Figure 18. Merging all local dense point clouds

Figure 18 combines all local dense point clouds successfully reconstructed from our method. This operation is seamless because objects are reconstructed locally and independently. Compared to stereo reconstruction, our method can now reconstruct objects that may get discarded after rectification. In other words, our method does not assume the camera heads to follow stereo vision or at the same height strictly. This is usually the case for real-life photos where different images of an object will likely be captured at random angles.

Another advantage of our method is that it can be easily generalized to the n -view setup. When searching for the optimal correspondence, we simply accumulate matching errors on $n - 1$ views. Moreover, the choice of the image where we sample the first point is not necessarily the front view. It could be the left or right view and the method still holds. This means that we can reconstruct the same object from different views and cleverly combine the point clouds, a subject known as bundle adjustment.

Finally, despite being relatively slow compared to stereo reconstruction, our method has a lot of room for parallel implementation. Specifically, the match at one sample point is completely independent of the others, making parallel executions feasible. In other words, our method's performance is comparable to the traditional stereo case given an appropriate implementation. Still, it demands texture-rich object surfaces, even if sophisticated matching is employed. This is the problem that encourages people to seek solutions from deep-learning-based approaches.

6. Point cloud reconstruction with DUST3R

DUST3R is a groundbreaking deep-learning approach in 3D reconstruction, notable for its ability to analyze image content directly, bypassing the need for camera parameters. This robust and fast method works with just a few images, even without overlap, and is the first to achieve both 3D reconstruction and visual localization without user intervention.

6.1. A brief overview of DUST3R

DUST3R is characterized as "unconstrained" due to its ability to tackle the challenge of 3D reconstruction from images without relying on camera parameters or viewpoint poses. Unlike traditional projective camera models, DUST3R formulates pairwise reconstruction as a regression of pointmaps. By efficiently decoding these pointmaps from pairs of images, DUST3R simplifies the extraction of detailed scene geometry, while still providing rich geometric details in the process.

The network architecture leverages standard Transformer encoders and decoders, utilizing existing powerful pre-trained models. This approach directly produces a 3D point cloud of the scene along with a depth map and relative and absolute camera poses.

6.2. State-of-the-art point cloud reconstruction

According to the authors of DUST3R, their method excels in three challenging scenarios. We evaluate its performance with our scene in these conditions. First, we test a single front-facing image (Figure 1), assessing monocular depth estimation. Second, we use images from nearly opposite viewpoints, which are the left and right views in Figure 1, to test wide-baseline stereo reconstruction. Finally, we combine three views to leverage the model's global alignment strategy, enhancing 3D reconstruction accuracy and completeness.

In the first case, with only one image, DUST3R reconstructs depth correctly. However, some 3D points for certain objects, such as the cauliflower, Ganesh, and half of the black angry bird's face, are missing (Figure 19 top). Some of these points are missing due to occlusion (e.g., the black angry bird), while others are due to the difficulty of the objects themselves. For example, the cauliflower's white color matches the background, and the Ganesh has complicated details and colors.

In the second case, we test the model with drastic viewpoint changes. As shown in Figure 19 middle, highly occluded objects, such as the mushroom and cauliflower, cannot be reconstructed. Additionally, the upper half of the black angry bird is lost. The large viewpoint changes result in more lost details and noise in the reconstructed scene. Incorporating a bridging view, like a front view, could improve the results.

With three views involved, in Figure 19 bottom, the full scene is reconstructed, although some small details are still missing. Unfortunately, Ganesh cannot be reconstructed in any case. Several factors contribute to DUST3R's failure to reconstruct certain objects and the missing points, even when they are not occluded. First, the input image width is limited to 512 pixels, affecting image quality. Higher-resolution images would contain more detail, leading to better reconstructions. Second, more views generally result in better reconstructions, but the viewpoints should not be too far apart, as seen in the noisy reconstruction in case 2. Additionally, objects should not be overly occluded in each photo. Finally, choos-



Figure 19. From top to bottom, the first image depicts scene reconstruction from the single front view, the second image from the significant viewpoint changes of the left and right view, and the last image involves all left-front-right views. Red circles show the positions of objects not reconstructed such as the mushroom and cauliflower in the second scene.

ing feature-rich scenes with distinct features or textures, like the red stuffed animal or black angry bird, improves results. Objects with bland colors (e.g., cauliflower) or complicated details (e.g., Ganesha) should be avoided. Some observations here are already made in Section 4 showing the shared nature of 3D reconstruction, regardless of the technique.

7. Comparisons and conclusion

Before ending, we will give a final comparison between the approaches taken in this project. We started with stereo vision, a relatively fast and widely adopted method based on image pair rectification. Given an appropriate choice of hyperparameters, a stereo setup provides a dense reconstruction of the whole scene, though some objects may get discarded if the stereo views greatly deviate. One may also need to remove noise in disparity maps, a common issue when applying block-matching algorithms.

For the 3-view setup, we have fewer parameters to tune and the method is resilient to the relative positions of the camera heads. The trade-off is the correspondences must be considered across all views, resulting in decreased performance in multi-view reconstructions. Moreover, the choice of the matching algorithm is crucial, and one may opt for more robust algorithms other than plain template matching. Nevertheless,

the method we discussed for the 3-view setup can be generalized to multi-view in this work, and the computations can be gracefully executed in parallel to speed up the process.

Deep-learning approaches to 3D reconstruction show their unique solution to the problem by eliminating the need for calibration. The previous two methods depend heavily on the calibration process using ChArUco patterns, a condition that does not always hold in real-life photos. Models like DUST3R provide a fast, accurate, and simple solution for 3D reconstruction, representing a considerable step forward in the 3D reconstruction field.

In the end, the choice of which approach to take boils down to the very specific condition of the input image. If input images closely match a stereo vision, stereo reconstruction should be used to reduce the cost. If instead there are many random views of the same object, multi-view reconstruction is more recommended. Finally, if the number of input images is relatively limited that the calibration cannot be done accurately without unique patterns, one may opt for deep-learning-based reconstructions to enjoy state-of-the-art results.

References

- [1] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [2] D. G. Lowe, “Distinctive image features from scale-invariant keypoints”, in *International Journal of Computer Vision*, Springer International Publishing, 2004, pp. 91–110. DOI: <https://doi.org/10.1023/B:VISI.000029664.99615.94>.
- [3] H. Hirschmuller, “Stereo processing by semiglobal matching and mutual information”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008. DOI: [10.1109/TPAMI.2007.1166](https://doi.org/10.1109/TPAMI.2007.1166).
- [4] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “Orb: An efficient alternative to sift or surf”, in *2011 International conference on computer vision*, Ieee, 2011, pp. 2564–2571.
- [5] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and M. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion”, *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, ISSN: 0031-3203. DOI: <https://doi.org/10.1016/j.patcog.2014.01.005>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320314000235>.
- [6] M. Aharchi and M. Ait Kbir, “A review on 3d reconstruction techniques from 2d images”, in *Innovations in Smart Cities Applications Edition 3*, M. Ben Ahmed, A. A. Boudhir, D. Santos, M. El Aroussi, and I. R. Karas, Eds., Cham: Springer International Publishing, 2020, pp. 510–522, ISBN: 978-3-030-37629-1.
- [7] Wang, Shuzhe and Leroy, Vincent and Cabon, Yohann and Chidlovskii, Boris and Revaud Jerome, *DUST3R: Geometric 3D Vision Made Easy*, 2023.